



Quality Strategy & Risk-Based Verification

SEN 381

Chanice Grant (601597)

Megan Wathen (601611)

Milenke Field (578122)



Table of Contents

INTRODUCTION	2
1. QUALITY STRATEGY & RISK-BASED VERIFICATION	2
1.1 Quality Foundations	2
1.1.1 Quality Assurance (QA)	2
1.1.2 Quality Control (QC)	2
1.1.3 Verification and Validation (V&V)	2
1.1.4 Relationship Between QA, QC and V&V	3
1.2 Risk-Based Verification Strategy	3
1.2.1 Risk-Based Verification	3
1.2.2 Prioritising Verification According to Risk	3
1.2.3 Application to CivicConnect	4
1.3 Verification Evidence	4
1.3.1 Unit Testing	4
1.3.2 Integration and API Testing	5
1.3.3 System and End-to-End Testing	5
1.3.4 Comparison of Verification Evidence	5
1.4 Automation and Quality Gates	6
1.4.1 Automated Verification	6
1.4.2 Continuous Integration Quality Gates	6
1.4.3 Benefits and Limitations of Automation	7
1.5 Critical Discussion	7
1.5.1 Why Passing Automated Tests Does Not Prove Overall Software Quality	7
1.5.2 Combining Automated and Manual Verification Evidence	8
1.6 CivicConnect Risk-to-Verification Evidence Map	8
1.6.1 Purpose of the Evidence Map	9
1.6.2 Risk-to-Verification Evidence Map	9
1.6.3 Interpretation and Recommended Verification Priorities	10
2. SECURITY ENGINEERING	11
2.1 Purpose of Threat Modelling	11
2.2 Security Concern Chains	12
2.3 Relating Secure Design, Secure Coding, and Dependency/Security Checks	13
2.4 Limitations of Relying on Automated and/or AI Security Tools	13
3. PRODUCTION READINESS, DEPLOYMENT & OPERATIONAL EVIDENCE	13
3.1 Production Readiness	14

3.2 Deployment Strategy	14
3.2.1 Environment Strategy.....	15
3.2.2 Secrets and Configuration.....	15
3.2.3 Release and Deployment	16
3.3 Operational Evidence	17
3.3.1 Failure and Recovery	17
3.3.2 Observability	17
3.4 Operational Quality	18
3.4.1 Reliability	18
3.4.2 Performance.....	18
3.4.3 Scalability	18
3.4.4 Cost.....	18
3.4.5 Supportability	19
3.5 CivicConnect Production-Readiness Evidence Matrix	19
3.6 Critical Discussion: Why Functional Success Does Not Prove Production Readiness	20
CONCLUSION	21
References	21
AI RESEARCH AND VERIFICATION RECORD.....	21

INTRODUCTION

1. QUALITY STRATEGY & RISK-BASED VERIFICATION

1.1 Quality Foundations

Quality of software must be viewed during the entire life cycle of software development and not just at the end during the software testing. Quality assurance, quality control, verification and validation are four distinct yet complementary methods of demonstrating confidence in the software meeting its requirements and are appropriate for its intended use. In addition, software quality evaluation can also be directly related to specific requirements and measurable criteria, where the requirements provide significant support in determining software quality expectations have been met or not (Verdugo, et al., 2025)

1.1.1 Quality Assurance (QA)

The focus of Quality Assurance (QA) is on the processes adopted in software development and on avoiding problems with quality. It includes setting up suitable practices and standards, reviews and quality activities in the software life cycle. QA works towards a controlled development process to minimize the chance of defects being introduced, rather than merely discovering them following the implementation.

The impact of QA on CivicConnect would thus affect the planning, development, review and verification of the system during the development process. This will help to make a more preventive effort towards quality rather than depending only on testing, which will take place towards the end of the project.

1.1.2 Quality Control (QC)

Quality Control (QC) is more direct with respect to the software product and its output. It is used to detect failures or deviations from established requirements and quality standards. Testing, inspection and review activities can generate evidence to confirm the behaviour of the implemented functionality.

In CivicConnect, QC evidence might, for example, result from tests of request-management functionality, from an examination of the implemented components, and from the examination of whether expected results meet with defined acceptance criteria. QA and QC therefore work on different aspects of the quality: QA is more interested in preventing the appearance of defects using appropriate processes, while QC is more interested in detecting the defects that made their way into the software.

1.1.3 Verification and Validation (V&V)

Verification and validation are complementary to software correctness and suitability evidence. Verification checks if software artefacts and implementation meet the required and designed expectations to ensure that the software is correct. Validation is an examination of whether a system, which is obtained, meets its intended use and stakeholder requirements.

The difference is significant since software might meet the specifications, but not what the users need. Thus, evidence for the verification task does not guarantee quality of software. The two viewpoints are relevant for CivicConnect: evidence of implementation in relation to the requirements, and evidence of the resulting behaviour in relation to the intended service-request processes.

1.1.4 Relationship Between QA, QC and V&V

Quality Assurance, Quality Control and Verification and Validation must not be seen as stand-alone processes. QA sets up the processes to ensure software quality, and QC looks for defects and deviations in the software. Verification and validation provide evidence to help decide if the software has been constructed in accordance with its specifications and if it meets its intended use.

1.2 Risk-Based Verification Strategy

1.2.1 Risk-Based Verification

In risk-based verification, based on the level of risk of different parts of the software system, the effort for verification is allocated. Instead of testing every function and type, more extensive testing is done on areas where there are more potential consequences. Risk can be assessed by considering factors such as the likelihood of a failure occurring and the impact if that failure occurs.

(Casola, et al., 2024) illustrate a risk-based approach in which risk is determined based on likelihood and impact, then security assessment and security testing are carried out based on the risk. This demonstrates that risk assessment can help to focus verification on those parts of the system where the evidence is likely to be less complete than needed rather than assuming all functions are equally critical.

Note that risk-based verification does not mean that lower risk functionality will be neglected. Rather, it offers a systematic approach to identify where testing time and resources are constrained and should be focused. This is especially significant when a development team cannot conduct all the possible verification processes in a consistent manner.

1.2.2 Prioritising Verification According to Risk

The probability and impact of failure, in conjunction with the business and technical significance of the impacted functionality, can be used to determine verification priorities. A function that has a high chance of failure, might need further testing, and/or a function that has a low chance of failure, but has a high consequence of failure, might warrant more testing.

The type of information needed, therefore, will be dependent on the type of risk. For instance, a unit test can give evidence that one component performs correctly by itself and may not give evidence that the multiple components interact correctly. Likewise, if an integration or API test is performed, there may be evidence of interactions between the components, but it does not necessarily mean that the whole system meets all the expectations of the various stakeholders.

In turn, a risk-based approach can incorporate multiple types of verification, as opposed to only a single testing method. This enables that the level of strength of the verification evidence and the type of evidence is in line with the risks addressed.

1.2.3 Application to CivicConnect

The findings of this research could help to shape the later development of CivicConnect by providing insights for the team on which behaviours of the system they would need to have more evidence of verifying. Possible areas of concern may be requesting submission, request assignment, request state changes, access to sensitive information, and request history. Outcomes of failure on these aspects may vary for the requester, staff and management.

For instance, if the request status is incorrect, it might result in misleading information being returned to the requester, and if a user of the request is not authorised to have access to that request's information, it may lead to very different outcomes. Similarly, not retaining the status history of a request could impact traceability and accountability. These examples illustrate the need to prioritize verification effort based on the probability, severity and business/technical criticality of different failures.

1.3 Verification Evidence

Various verification methods are used to give evidence for various aspects of software behaviour. There is no single technique that shows the overall quality of software, since every technique identifies different types of failure at different levels. A more robust verification approach will then be the integration of different types of evidence. For a modern web-based system like CivicConnect, there are three types of verification evidence that can be helpful: unit testing, integration/API testing and system/end-to-end testing.

1.3.1 Unit Testing

Unit testing is applied to testing the individual units or components of software program separately. It is meant to verify the correctness of a certain portion of the implementation under certain conditions. It is possible to get instant and repeatable feedback during development using unit tests and these are especially helpful for identifying individual method, class or business rule errors.

In a system like CivicConnect, it may also be possible to rule out the need to run the whole application for unit testing to check for individual behaviours like a validation rule, status transition logic or calculation. This can enable the developers to detect defects near the component where they are created.

Independent testing, however, will not prove that independently tested components will interact correctly when they are combined, as will be required for successful unit tests. They also can't guarantee the entire application meets user expectations or acts as it does in the environment in which it is deployed. Therefore, for unit testing to be used as verification evidence, it is important that it is supplemented by testing at higher levels.

1.3.2 Integration and API Testing

Integration testing checks whether components or services behave correctly when working together, and API testing can check the behaviour of interfaces that are used to exchange information between the components. This is especially true for modern web applications in which functionality can rely on the exchange of information between layers of an application, databases or external services.

(Golmohammadi, et al., 2023) present several concerns and techniques to be addressed in testing RESTful APIs to evaluate their behaviour. This is why is API testing useful to detect failure in such a situation where failure may not be visible when testing individual components.

CivicConnect would consider evidence of potential integration as sending/receiving information about requests via an API, pulling request information from saved data, and exchanging data between application components. This could reveal problems involving incorrect data transfer, interface contracts, validation or error handling.

However, API or integration tests cannot guarantee the full user flow is working correctly by themselves. They may also not be able to detect usability issues or user behaviour that can only be found when all components of your application are used together. Integration/API testing should be used in conjunction with other verification methods.

1.3.3 System and End-to-End Testing

System and end-to-end tests check behaviour on a more extensive application flow. This type of testing can evaluate the performance of several components of the system to enable a desired user, or business process, to function.

The service request might be submitted, assigned, tracked, resolved and returned to CivicConnect as an end-to-end scenario. This may offer the proof that the components take part in a foreseen operation chain and can make evident problems which are not observed when the components are examined individually.

End to end testing is not enough by itself, however. When a complete workflow fails, it's harder to determine which part of the workflow failed, compared to focused unit or integration tests. End-to-end testing also cannot be practically exercised to cover all the input, internal path or failure conditions. It therefore offers more general behavioural evidence but should be complemented by more specific intervention-based evidence at lower levels.

1.3.4 Comparison of Verification Evidence

The three verification methods are complementary since they are considering the software at distinct levels. Whilst Unit testing gives focused information regarding individual components and Business rules, Integration/API testing assesses how information and interactions are passed between components. System/end-to-end testing gives more confidence that several components of the application are integrated to enable desired workflows.

It's also worth noting their restrictions. Passing unit tests does not mean successful integration, passing integration tests does not mean successful complete user workflows, and passing end to end tests does not mean complete user workflows are successful by every underlying component, edge case or failure condition. Thus, totally depending on a single type of testing might overlook vital software risks.

These differences can be used in a risk-based verification strategy to decide on the combination of evidence for a specific risk. If the functionality is more critical, then it could be more extensive and/or evidence from multiple levels of verification could be applied, while functionality that is not as critical may be less extensive with fewer levels of verification. The goal is not to test as much, but rather to gather verification evidence that is proportional to the probability, consequence and severity of the failures under consideration.

1.4 Automation and Quality Gates

Selected checks can be more reliably carried out in an automated manner and repeated throughout the development process, which can help reinforce a verification strategy. This is valuable in the case of software that undergoes frequent changes, since previously tested behaviour can be tested again without an equivalent amount of manual work. But automation should provide supporting evidence and not be taken as a guarantee that there are no defects in the software.

1.4.1 Automated Verification

Automated verification is the process of running tools/scripts that perform a defined group of checks and compare the output of the software to a desired outcome. The tests – whether Unit

tests, Integration/API tests or selected system tests – can all potentially be automated to enable verification activities to be repeated as the software progresses.

One of the major benefits of automated verification is that it provides immediate feedback. If the software is changed by the developer, automated tests can tell if any previously working behaviour has been impacted. This is where automation can be especially helpful for regression testing, and when frequent changes occur in iterative development.

Software test automation research also reveals potential problems in the process that can impact the effectiveness of software automation testing. (Abrar, et al., 2025) explore various issues involved with the automated software testing and conclude that the successful implementation of automated software testing requires more than just the introduction of software testing tools. Therefore, how effective automated verification is will rely on the quality, relevance, and effectiveness of the tests being performed and the effectiveness of incorporating automated verification into the development process.

In CivicConnect, automated verification might be used to regularly validate significant operations like validation rules, transition in request statuses, or interactions between selected components. The actual automated tests to use would depend on the identified risks and verification priorities as identified in the subsequent phase of development.

1.4.2 Continuous Integration Quality Gates

Continuous Integration (CI) is a technique that enables developers to make changes and test them regularly. Automated verification can be a part of this process, which can include running certain checks after changes are implemented. It is from there that a quality gate can define conditions which must be met before a change is permitted to move any further in the development process.

Examples of evidence used within a quality gate could include successful automated tests, build results or select static checks. These mechanisms can enable the developers to receive feedback early and minimize the chances of known failures getting through unawares.

For CivicConnect, changes might be assessed by using the CI quality gates before being accepted to the controlled code base. For instance, if a change is proposed, it may run the appropriate automated tests and give the reviewers evidence to consider before approving the change. This research assignment would not be determining the specific quality gates and acceptance thresholds but rather would be explained during the later engineering milestone.

Importantly, a successful CI pipeline tells you only that the checks that you've set up in the CI pipeline have passed. It does not prove that all the pertinent quality attributes, requirements, user scenarios and failure conditions have been assessed.

1.4.3 Benefits and Limitations of Automation

There are several potential advantages to automated verification, such as repeatability, quicker feedback, and the ability to run a chosen test repeatedly as software evolves. It can save repetitive manual effort and help to test regression – testing to make sure that any existing function still works when changes are made to it. Automation can also generate consistent proof that can be saved as component of the advancement and review procedure.

But there are disadvantages when it comes to automation. Automated tests are only designed to check behaviours and conditions. Inappropriate, incomplete or obsolete tests can thus test fine but contain serious flaws. Automation might not be the best approach for quality attributes that depend on human decision making, such as usability or if a workflow fulfils the requirements of its users.

Therefore, an automated test suite (or CI Pipeline) that is green does not necessarily mean software quality. Automated verification should be part of a comprehensive approach that incorporates various forms of verification and considers the risks each type of evidence can and cannot mitigate. The difference between these is significant when considering if there is enough evidence for a system or change to advance towards release.

1.5 Critical Discussion

While automated verification can give a wealth of repeatable evidence of software behaviour, it should not be taken as an indicator of software quality. The trustworthiness of automated evidence is dependent on what is being tested, how well the tests cover-relevant risks and if there are important quality issues not addressed by the automated test suite.

1.5.1 Why Passing Automated Tests Does Not Prove Overall Software Quality

If a test is successful, this means that the software created the desired output under the specific circumstances that the test was applied. Does not show that all possible inputs, processes, failure conditions or quality attributes have been tested. Thus, a system can pass its entire automated test suite but have defects or weaknesses that the tests didn't know to check for.

According to (Abrar, et al., 2025), software test automation has its share of challenges, which further highlights that the efficacy of software test automation is dependent on how it is designed, implemented and maintained. A complete and effective verification can therefore not be assumed from the mere presence of automation.

Another side effect of automated tests is that they can give a false sense of security because test coverage is seen as a measure of software quality. Many successful passes can show a wide range of exercising of predefined scenarios, but it does not necessarily mean that the most critical scenarios have been addressed. The test suite may not cover important edge cases, unexpected interactions and changing requirements of the system.

This is particularly significant to a system like CivicConnect. Automated tests might only be able to show that a request status changes properly given certain conditions, but not that the entire workflow is user-friendly, or that an unusual series of actions leads to an untested failure. The green automated pipeline is thus a form of verification evidence, not a release decision in and of itself.

1.5.2 Combining Automated and Manual Verification Evidence

A robust verification strategy is developed by leveraging automated evidence and using the right type of manual evidence. Automated testing is useful in cases of repetitive and deterministic tests which must be performed often. This can be supplemented by manual verification which

involves looking at behaviours that need to be explored, understood in context or clarified by human judgement.

For instance, predefined rules for how requests are processed can be efficiently tested using automated tests, as can API responses and regression scenarios. Manual exploratory testing might explore unexpected user behaviour or combinations of actions different than those anticipated when automated tests were written. Human evaluation can also be used to obtain evidence of factors like usability and understandability of the system from the user's point of view.

These methods are not being combined to make manual testing the superior approach to testing. Rather, every type of evidence should focus on potential risks that it can reveal. Automated verification provides repeatability and quick feedback, but manual can be flexible and exploratory if there are not enough predetermined automated checks.

The overall strength of the evidence available, not just the number of passed tests, should thus be assessed using a risk-based quality approach. Where there is a higher level of risk for functionality, information from multiple complementary verification activities may be needed to reach a level of confidence. This ensures a better case for "quality" and "release" decisions than the notion that a successful automated testing suite means it is ready for use.

1.6 CivicConnect Risk-to-Verification Evidence Map

1.6.1 Purpose of the Evidence Map

A Risk-to-Verification Evidence Map serves as a structured mapping of software risks found and evidence that can be used to assess software risks. It aims at ensuring activities are conducted for verification for a clear and relevant quality issues and not just because the test exists or is easily mechanised.

For CivicConnect, the map can identify potential failures in key behaviours of the system and what their potential impact might be, and what evidence of verification would be appropriate, and what decision would it inform. This aligns with the risk-based verification concepts described in Section 1.2, that more robust or multiple types of evidence may be needed to demonstrate more impactful or more critical functionality.

1.6.2 Risk-to-Verification Evidence Map

<i>Risk / Quality Concern</i>	<i>Potential Consequence</i>	<i>Verification Evidence</i>	<i>What the Evidence Would Demonstrate</i>	<i>Possible Progression / Quality Decision</i>
Incorrect request status transition	Requests may enter an invalid state or display incorrect progress to users	Unit tests + integration/API tests	Business rules reject invalid transitions, and accepted changes are correctly	Do not progress the affected functionality while critical transition

			processed between components	failures remain unresolved
Request update only partially persisted	Current request information and its history may become inconsistent, reducing traceability	Integration tests + database/persistence verification	Related updates are stored consistently, and failures do not leave incomplete records	Require successful integrity evidence before the persistence functionality progresses
Unauthorised access to request information	Sensitive request information may be exposed to users without appropriate access	Authorisation-focused integration/API tests + manual negative testing	Restricted operations and information cannot be accessed through tested unauthorised scenarios	Access-control failures affecting sensitive information should prevent progression until addressed
Request submission workflow fails between components	A requester may believe a service request was submitted when it was not successfully processed	Integration/API tests + end-to-end tests	Information is transferred correctly and the complete submission workflow reaches the expected outcome	Progress only when the critical submission path operates successfully across the required components
Previously working behaviour breaks after a software change	Existing CivicConnect functionality may regress as new changes are introduced	Automated regression tests + targeted manual verification	Previously verified behaviours continued to operate, and important changed areas have been examined	Regression failures should be investigated before accepting the change
User-facing workflow is technically functional but difficult or confusing to use	Users may make errors or fail to complete required service-request activities	End-to-end testing + manual/user-oriented evaluation	Complete workflows operate and their practical interaction can be evaluated beyond technical correctness	Usability concerns should inform whether further refinement or evidence is required before release

1.6.3 Interpretation and Recommended Verification Priorities

The evidence map illustrates how varied CivicConnect risks are in terms of what they need to be verified against. No one testing technique can provide enough evidence for each quality

concern. The risks of communication or persistence aspects of the component may be effectively investigated using unit tests, while risks of communication or persistence between components need integration-level evidence. End-to-end testing and manual evaluation may be required for complete workflows, depending on the concern, and there may be concerns involving human interaction.

Verification priority should also be based on the risk of failure. Potential integrity, access and service request loss or processing issues could have a significant impact on the reliability and trust of the system, as well as its confidentiality and traceability. Therefore, these areas would merit more verification evidence than would functionality with less significant consequences.

The map also shows the importance of complementary evidence. For instance, if the request-status rules pass the unit tests, the tests would not show that the status change is preserved and correctly propagated throughout the system. Unit and integration and, if possible, end-to-end evidence together gives more confidence as the same important behaviour is considered from different angles.

These priorities are not final CivicConnect verification decisions, but rather research-informed recommendations. The team could then apply the risks identified as a basis for selecting specific tests, defining acceptable criteria for progress and establish connections between verification evidence and requirements and implementation artefacts in the later engineering milestone.

2. SECURITY ENGINEERING

Security in a software engineering project should be looked at early in the software development lifecycle (SDLC). Since issues in security could not only lead to data breaches and consequently the loss of trust, but they also can impact the cost of a project, time required to complete, architecture setup (Odera, et al., 2023). Thus, security it should be considered from the start as a continuous responsibility and not just a last-minute check to avoid technical debt (Alenezi & Almuairfi, 2019).

2.1 Purpose of Threat Modelling

Threat modelling is used to identify potential exploit pathways before the system architecture is locked in by uncovering security design flaws (Microsoft Ignite, 2019) Threat modelling aims to connect a threat to a security requirement/control objective, and then to a specific engineering control, and then finally to a form of verification evidence. However, threat modelling extends beyond its core function of threat identification to cultivate a culture of security awareness for different organizations (Wiersema, 2025). There are many different threat models available (i.e. STRIDE, VAST, Attack Trees, etc.) but currently STRIDE will be analysed (Shevchenko et al., 2018).

STRIDE functions as an active architectural decision-support tool since it informs trust decision boundary decisions between a browser and a webserver for example or between an app and a third-party API. More specifically, when STRIDE is used a software system it is deconstructed into its individual components (i.e. database, API). Those components are then taken through the six STRIDE components (Spoofing, Tampering, Repudiation, Information Disclosure, Denial

of Service, Elevation of Privilege) to determine specific attack methods, paths, or scenarios attackers can use to exploit a vulnerability within a specific software component (Milic, et al., 2025). After the threat is identified, it can be associated with a specific security requirement/control objective such as authentication, integrity, nonrepudiation, confidentiality, availability, and authorization (Microsoft Ignite, 2019). Which can in turn point to matching engineering controls used to counter the threat.

The different ways of countering risk include reduction, avoidance, sharing, transfer, and acceptance (IBM, 2026). With regards to risk reduction, STRIDE can show decision makers what exact elements need fixing. For example, STRIDE analysis can lead to the decision to incorporate multi-factor authentication (MFA) to reduce the risk (via mitigation) that arises from a spoofing threat to do with a login page. In terms of risk avoidance, when STRIDE reveals a threat too costly to fix, it can allow decision makers to safely choose to remove a troublesome feature. Pertaining to sharing risk, STRIDE can identify complex threats that may require multi-party risk management between the client and product provider. Moreover, it can help decision makers realize how complex some risks can be and decide which responsibilities they must handle locally to lessen the risk and which responsibilities must be implemented externally by the clients for effective risk management. With respect to risk transfer, STRIDE can provide cyber insurance providers with a clear understanding of a company’s risk profile before they make important decisions like policy approvals. If the threat must be accepted, STRIDE at least provided a structured list and processing of the threats beforehand to make an informed, deliberate, and documented decision instead of making an ad hoc decision and dealing with major repercussions afterwards.

Finally, verification evidence (penetration test report, code review records, logs, etc.) can be produced to see if the engineering control was implemented and satisfies the specific security requirement/control objective.

2.2 Security Concern Chains

Security concerns originate from proactive and reactive risks that are observed and associated with loss or damage (Alenezi & Almuairfi, 2019).

Threat/ Misuse	Security Requirement/Control Objective	Possible Engineering Control	Verification Evidence	Residual Risk/Limitation
Credential Stuffing	Authentication	Enforce strong authentication methods, enforce secure password creation and handling, session management, incorporate authentication controls	‘-‘ login tests, security reviews, authentication tests, session/security testing	Testing cannot guarantee that all possible credential attacks have been prevented

Insecure Direct Object Reference (IDOR) or Broken Object Level Authorization (BOLA)	Authorization/Access Control, Sensitive Info/Privacy	Least-privilege design, make use of server-side authorisation checks	API/security testing, '+/-' authorization tests	Incorrectly identified roles may still be present
Secrets Detection/ Exposure	Secrets/ Configuration	Do not insert secrets into source code	Secret scanning, deployment inspection, repo inspection, configuration review	Scanners might miss secretes stored in unusual forms
Input Injection	Input/Data Handling	proper error handling, server-side validation, safe data access practices	Input validation test results, security testing results, code reviews, '-' API tests	Testing cannot guarantee protection against all malicious inputs, framework/library vulnerabilities can be sources of more risk

2.3 Relating Secure Design, Secure Coding, and Dependency/Security Checks

Secure design can be considered as the first step in developing a secure software development lifecycle (SSDLC), it spans over the planning and design phases in SDLC. It considers integrating security from the start of a project by looking at architecture, threat modelling, security requirements, secure design patterns, and component isolation ((Mbata, 2026); (Washington, 2025)). Overall, secure design focuses on planning how threats will be avoided and how to reduce vulnerabilities (Washington, 2025). The next step is building the secure system which involves secure coding. In SDLC, this step implements by taking the secure design and transforming it into secure and resilient code (Washington, 2025). More specifically it deals with input validation, error handling, authentication and authorization, memory management, and handling secure libraries.

Proper secure design and careful secure coding together contribute to a system that is resilient towards cyberattacks is known as secure coding (IBM, 2026). One cannot expect to have a secure software product at the end if one has a well-planned but poorly implemented system. Likewise, securely implemented code cannot substitute a poorly planned design to start with as a design determines the architecture of the secure coding that should be executed because of the planned design.

Dependency/Security checks confirm whether the secure code is secure enough or not by using both authenticated and unauthenticated scans as well as document tracking (Tchami, 2023). These checks scan for vulnerabilities introduced in the system due to insecure design or coding

practices. Without dependency/security checks the final deployed product may still be insecure due to complex vulnerabilities that may have escaped/have been masked by falsely identified secure validation of the first two steps. Thus dependency/security checks can validate the presence of security incorporated into both design and source code to confirm that a software system can be categorized as secure (Tchami, 2023).

2.4 Limitations of Relying on Automated and/or AI Security Tools

Automated decisions are decisions that remove the end-user responsibility and instead give systems, remote services, and organizations the power to establish policies that are automatically enforced. Many companies prefer using automated security tools over manual security options to avoid errors made by end users (Edwards et al., 2008). While automated and artificial intelligence (AI) security tools may be faster and potentially more accurate (with proper fine tuning) than manual checks, they are still not fool-proof.

Take vulnerability scanning for example. Although vulnerability scanning can be useful, automated vulnerability scanning does not have the ability to spot all vulnerabilities present in a system. Holm et al. (2011) found that authenticated vulnerability scanning can be used to scan for weak points in a system, but they also noticed that scanning results are inconsistent since their research pointed out that scans done on Windows operating systems (OS) are more accurate in detecting vulnerabilities than scans done on Linux systems. Thus, accuracy can be a limitation of using automated security methods. Thomson et al. (2024) agrees that incorporating AI methods in automated vulnerability testing can enhance workflows. However, Thomson et al. (2026) also recommends cautious AI adoption as their research observed that although reliability and adaptability improve with the use of AI, unfortunately the end-users responsible for security decisions became less alert to observing errors that still arise from AI suggested security recommendations due to response bias developed from the habit of overriding several AI decisions. Even though this is not necessarily a technological limitation, the technology has influenced the decision-making skills of the end users accountable for making responsible security decisions for the current software system that is being developed (Thomson et al., 2024).

Azaz and Farwa (2026) further state that vulnerability scanners consistently have troubles with system overloading, poor prioritization and limited adaptability since they run on predefined checks that only allows them to detect weaknesses they have been programmed to find. The limited adaptability issue is likely why these scan results can be so inconsistent by detecting a varied number of weaknesses among different OS's. Azaz and Farwa (2026) then go on to suggest that adopting machine learning (ML) based methods can reduce the amount of false positives and negatives associated with automated vulnerability testing and thus improve reliability and adaptability of these tests.

There are several other limitations but only looking at these two fundamental and very basic limitations makes it obvious that entirely replacing manual end user security planning and problem solving with computer automated decision allowance may complicate efficient security tool utilisation more than aiding it. This is because computer automated systems still need to be monitored and maintained by humans, thus the more automated decisions are

made the more monitoring and maintenance will be required by humans (Bainbridge, 1983). Humans make mistakes and humans made and still must maintain computers so putting complete trust in automated and AI-enhanced security methods is an unwise security practice.

3. PRODUCTION READINESS, DEPLOYMENT & OPERATIONAL EVIDENCE

3.1 Production Readiness

Production readiness refers to the extent to which a software system is prepared to operate reliably, securely and supportably in its intended production environment. A system should therefore not be regarded as production-ready merely because it compiles successfully, passes functional tests or works on a developer's machine. Production readiness requires evidence that the system can be deployed consistently, configured safely, observed while running and recovered when failures occur.

A production environment introduces conditions that may not exist during development. These can include different infrastructure, configuration settings, workloads, security requirements, dependencies and operational constraints. Differences between development, testing and production environments can therefore create deployment risk if they are not understood and controlled.

A useful environment strategy generally distinguishes between **development**, **test or staging**, and **production** environments. Development environments support active implementation and debugging, while test or staging environments allow changes to be assessed under conditions that resemble production more closely. Production contains the live system used by actual users. Maintaining reasonable parity between these environments reduces the risk that software behaves differently after deployment (Microsoft, 2025).

This does not mean that every environment must be identical. Production may require stronger security, larger infrastructure or different scaling arrangements. The important requirement is that differences should be **intentional, documented and controlled**, rather than accidental.

Microsoft's Azure Well-Architected Framework emphasises standardised development processes, observability and safe deployment practices to reduce process variation and operational disruption (Microsoft, 2025).

For CivicConnect, this research suggests that later production-readiness decisions should consider whether the application's architecture, configuration, dependencies and data requirements can behave consistently across environments. Assignment 3 should not yet

select the final CivicConnect hosting or environment configuration; those decisions should be made later using the project's actual constraints and implementation evidence.

3.2 Deployment Strategy

A dependable deployment strategy should make software releases **repeatable, controlled and traceable**. Successful compilation or packaging does not itself demonstrate successful deployment because the application must also operate correctly with the target environment, configuration, infrastructure and dependencies.

Google's Site Reliability Engineering guidance describes reliable release processes as reproducible and automated rather than relying on unique manual procedures. Repeatability reduces accidental differences between releases and makes deployment behaviour easier to understand and audit.

3.2.1 Environment Strategy

A software system normally progresses through different environments before it reaches production. A development environment is primarily used by developers to implement and debug functionality, while a test or staging environment provides a controlled location in which the behaviour of the system can be evaluated before release. The production environment is the live environment in which the software is used by its intended users.

Separating these environments reduces the risk of unfinished or unverified changes affecting live users. However, separation alone does not guarantee reliable deployment. Significant differences between environments can create failures that were not visible during development or testing. For example, differences in configuration, database versions, external dependencies, permissions or network settings may cause an application that worked correctly during development to behave differently after deployment.

Environment parity therefore aims to keep important environmental characteristics reasonably consistent while allowing legitimate differences such as production security controls, capacity and scaling. Differences between environments should be intentional, documented and controlled rather than accidental (Microsoft, 2025).

For CivicConnect, this research indicates that later production-readiness decisions should consider how development, testing and production environments can provide sufficiently representative conditions for verification. Assignment 3 does not determine the specific CivicConnect hosting or environment configuration; this remains a later engineering decision based on project requirements and implementation evidence.

3.2.2 Secrets and Configuration

Application configuration often differs between development, testing and production environments. Configuration may include database connection information, external-service

addresses, feature settings and other values needed for the application to operate. Sensitive configuration, such as passwords, API keys, authentication secrets and private credentials, requires stronger protection because unauthorised disclosure could compromise the system.

Sensitive values should therefore be separated from application source code. Hard-coding credentials creates unnecessary risk because source code is commonly copied between developer machines, repositories, build systems and deployment environments (OWASP Foundation, n.d.). If a secret is committed to source control, removing it from the latest version does not necessarily remove it from repository history.

Production configuration should instead be supplied through controlled mechanisms appropriate to the deployment environment. Access should be restricted to authorised processes and people, and changes should be managed so that configuration remains traceable and reproducible. This also allows the same application build to operate across different environments without embedding environment-specific secrets directly into the code.

For CivicConnect, the research suggests that later deployment planning should distinguish ordinary application configuration from sensitive secrets and determine how each will be supplied and protected. The specific configuration or secret-management mechanism should be selected during the later engineering milestone rather than being predetermined in Assignment 3.

3.2.3 Release and Deployment

A successful software build only demonstrates that the application's source and dependencies could be processed into an executable or deployable artefact. It does not prove that the software can be successfully deployed and operated in its intended environment. Deployment additionally depends on infrastructure, configuration, dependencies, networking, data compatibility and runtime conditions.

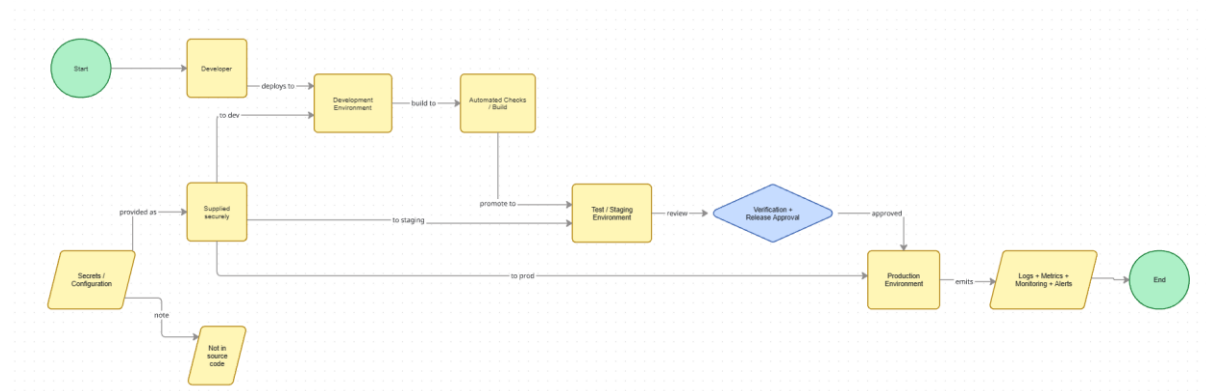
A reliable deployment process should therefore be repeatable and controlled. Repeatable deployment reduces dependence on undocumented manual steps and allows teams to reproduce how a particular version of the system was introduced into an environment (McNutt, 2016). Automation may support this process by carrying out defined build, verification and deployment activities consistently, although automation itself does not guarantee a successful release.

Release controls can also determine whether a version should progress toward production. Evidence may include successful verification results, configuration checks, deployment checks and approval of identified risks. These controls help distinguish the technical ability to deploy software from the engineering judgement that the release has sufficient evidence to progress (McNutt, 2016).

A deployment may technically complete while the resulting product is still unusable. For example, the application may start successfully but be unable to access its database,

communicate with another required service or process realistic workloads. Production readiness therefore requires evidence that the deployed system is functioning correctly in its intended environment rather than relying only on a successful build or deployment command.

In the CivicConnect context, this research can later inform decisions about what deployment process, release controls and evidence should be required. Assignment 3 should not yet select the final CivicConnect deployment platform or release mechanism.



3.3 Operational Evidence

3.3.1 Failure and Recovery

Production readiness requires consideration of what happens when a deployment or system component fails. A deployment process should therefore not only describe how a new version is introduced, but also how the system can recover if that version causes unacceptable problems.

One possible recovery approach is rollback, where a previous known-working application version is restored. However, rollback becomes more complicated when a release changes persistent data or database structures. An older application version may no longer be compatible with data produced or modified by the newer version. Consequently, application releases and database changes should be considered together when recovery strategies are evaluated (Microsoft, 2026).

Backups provide another form of recovery evidence. Merely creating backups does not demonstrate recoverability; teams also need confidence that backups are valid and can be restored within acceptable conditions. Production-readiness evaluation should therefore consider backup procedures, restoration evidence and the potential effect of data loss or downtime.

For CivicConnect, these findings suggest that later engineering decisions should consider how application versions, request data and database changes could be recovered following failure. Assignment 3 does not determine the final rollback or backup mechanism but identifies recovery evidence as an important factor in later production-readiness decisions.

3.3.2 Observability

Once software is operating in production, teams require evidence about its actual behaviour. Observability supports this by providing information that can help developers and operators understand the state of a running system.

Logs record events and activities that occur while the application runs. They can support troubleshooting by providing information about errors, requests and important system events. However, excessive or poorly structured logging can create large volumes of information without necessarily helping diagnose problems.

Metrics provide numerical measurements of system behaviour over time. Examples may include response times, error rates, resource utilisation and request volumes. Metrics can reveal trends or abnormal behaviour that may not be obvious from individual log entries.

Monitoring evaluates operational information so that the health and behaviour of the running system can be observed. Monitoring may use logs and metrics to detect conditions such as increasing failures or degrading performance (Frame et al., 2018).

Alerts provide notification when monitored conditions require attention. Effective alerts should represent meaningful conditions that require investigation rather than simply producing notifications for every unusual event.

Collecting operational data alone is therefore insufficient. Logs and metrics only become useful operational evidence when they can be interpreted and connected to meaningful system conditions and appropriate responses.

For CivicConnect, later engineering work could determine which system behaviours require logging, which operational metrics are significant and which conditions should trigger investigation. Those specific choices remain later project decisions.

3.4 Operational Quality

3.4.1 Reliability

Reliability concerns the ability of a system to provide expected service consistently over time. A functionally correct system may still be unsuitable for production if it fails frequently, loses information or cannot recover effectively from faults. Production-readiness evidence should therefore consider failure behaviour, recovery and continued availability of important functions. Reliability therefore includes not only availability but also resilience and the ability to recover from failure (Microsoft, 2026).

3.4.2 Performance

Performance concerns how effectively the system responds under expected operating conditions. Functional testing may prove that an operation produces the correct result but does not necessarily demonstrate that it produces that result quickly enough under realistic workloads. Response time, throughput and resource utilisation may therefore influence production-readiness decisions.

3.4.3 Scalability

Scalability concerns how the system responds when demand grows. A system that performs acceptably for a small number of users may experience degraded performance as request volumes, stored data or concurrent users increase. Scalability should therefore be evaluated according to realistic expected demand rather than assuming that successful development testing demonstrates sufficient capacity.

3.4.4 Cost

Cost is also an operational quality concern because increased performance, redundancy and scalability may require additional infrastructure or managed services. Production readiness therefore involves balancing technical capability against available financial resources rather than treating maximum infrastructure capacity as inherently preferable. Production-readiness decisions may consequently require trade-offs between reliability, performance, scalability and operating cost (Microsoft, 2025).

3.4.5 Supportability

Supportability concerns how effectively the system can be maintained and diagnosed after deployment. Clear operational documentation, meaningful logs, understandable configuration and identifiable failure information can reduce the effort required to investigate and resolve problems. A system that is difficult to diagnose or recover may be operationally unsuitable even when its main functionality works correctly.

These operational qualities involve trade-offs. Greater reliability or scalability may increase cost and operational complexity, while minimising cost may limit available capacity or redundancy. Production-readiness decisions should therefore evaluate operational qualities against actual system risks, stakeholder expectations and project constraints rather than attempting to maximise every quality characteristic simultaneously.

3.5 CivicConnect Production-Readiness Evidence Matrix

Production-readiness concern	Risk if ignored	Evidence before release	Evidence to observe after release
Environment consistency	Undetected differences may cause production-only failures	Documented environment/configuration differences and successful staging verification	Environment/configuration-related failures and deployment incidents
Secrets & configuration	Credentials may be exposed or incorrect	Evidence that sensitive values are separated from source code and configuration is validated	Authentication/configuration errors and evidence of unauthorised-access attempts

	configuration may compromise the system		
Repeatable deployment	Manual or inconsistent deployment may introduce errors	Repeatable deployment procedure, build evidence and deployment verification	Deployment success/failure history and release incidents
Rollback & recovery	A failed release may leave the service unusable	Tested recovery/rollback approach and identified recovery conditions	Recovery time, failed-release incidents and effectiveness of recovery actions
Backup & data recovery	Request information may be permanently lost or inconsistent	Backup evidence and evidence that restoration can be performed	Backup success/failure, restore events and recovery outcomes
Observability	Failures may remain unnoticed or be difficult to diagnose	Defined logging, metrics, monitoring and meaningful alert conditions	Error rates, operational logs, metrics, alerts and incident patterns
Performance & scalability	The system may become slow or unavailable as demand increases	Performance/load evidence under representative conditions	Response times, throughput, resource utilisation and changing demand
Supportability & cost	The system may become expensive or difficult to maintain	Operational documentation, support responsibilities and estimated resource requirements	Operational/support workload, infrastructure usage and ongoing cost

The matrix demonstrates that production readiness cannot be established through a single test or deployment result. Different concerns require evidence at different stages. Pre-release evidence can reduce the likelihood of known deployment and operational failures, while post-release evidence provides information about the behaviour of the system under actual operating conditions. The specific evidence thresholds and production controls used by CivicConnect remain decisions for the later engineering milestone.

3.6 Critical Discussion: Why Functional Success Does Not Prove Production Readiness

Software that passes functional testing and operates successfully on a developer's machine may still be unsuitable for production because functional correctness represents only one aspect of operational readiness. Production introduces different configurations, workloads, infrastructure, dependencies, security conditions and failure scenarios. A system may therefore provide correct outputs during development but fail because of incorrect production configuration, insufficient capacity, unavailable dependencies, incompatible database changes or inadequate recovery mechanisms. In addition, functional testing does not necessarily provide evidence of reliability, performance, scalability, observability, recoverability or supportability. Production readiness therefore requires a broader body of evidence demonstrating not only that the system functions, but that it can be deployed, operated, monitored and recovered safely under realistic conditions.

CONCLUSION

References

- Abrar, M. F., Majeed, M. F., Saqib, M. & Alferaidi, A., 2025. A data-driven analysis of software testing automation challenges using structural equation modeling (SEM) approach. *IEEE Access*, Volume 13, pp. 96159-96181.
- Alenezi, M. & Almuairfi, S., 2019. Security Risks in the Software Development. *International Journal of Recent Technology and Engineering (IJRTE)*, 8(3), p. 1.
- Bainbridge, L., 1983. Ironies of Automation. *International Federation of Automatic Control*, 19(6), p. 1.
- Casola, V., De Benedictis, A., Mazzocca, C. & Orbinato, V., 2024. Secure software development and testing: A model-based methodology. *Computers & Security*, Volume 137, p. 103639.
- Frame, J., Lenton, A., Thurgood, S., Tolchanov, A. & Trdin, N., 2018.** Monitoring. In: B. Beyer, N.R. Murphy, D.K. Rensin, K. Kawahara & S. Thorne, eds. *The Site Reliability Workbook*. O'Reilly Media. Available at: [Google SRE – Monitoring](#) [Accessed 22 September 2026].
- Golmohammadi, A., Zhang, M. & Arcuri, A., 2023. Testing restful apis: A survey. *ACM Transactions on Software Engineering and Methodology*, 33(1), pp. 1-41.
- IBM, 2026. *What is risk management?*. [Online]
Available at: <https://www.ibm.com/think/topics/risk-management> [Accessed 24 Sept 2026].
- IBM, 2026. *What is Secure Coding?*. [Online]
Available at: <https://www.ibm.com/think/topics/secure-coding> [Accessed 25 Sept 2026].
- Mbata, B., 2026. *LinkedIn: Security Shouldn't Be the Final Step in Software Development*. [Online]

Available at: https://www.linkedin.com/posts/bernard-mbata-1a9b0a200_cybersecurity-ssdlc-appsec-share-7495113202993680384-WUnZ/
[Accessed 23 Sept 2026].

McNutt, D., 2016. Release Engineering. In: B. Beyer, C. Jones, J. Petoﬀ & N.R. Murphy, eds. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media. Available at: [Google SRE – Release Engineering](#) [Accessed 22 September 2026]. The Google-hosted book confirms the 2016 publication and editors.

Microsoft Ignite, 2019. *Threat Modelling: Uncover Security Design Flaws Using The STRIDE Approach*. [Online]
Available at: <https://learn.microsoft.com/en-us/archive/msdn-magazine/2006/november/uncover-security-design-flaws-using-the-stride-approach>
[Accessed 24 Sept 2026].

Microsoft, 2025. *Operational Excellence design principles – Azure Well-Architected Framework*. Microsoft Learn. Available at: [Microsoft Operational Excellence guidance](#) [Accessed 22 September 2026].

Microsoft, 2026. *Design review checklist for Reliability – Azure Well-Architected Framework*. Microsoft Learn. Available at: [Microsoft Reliability checklist](#) [Accessed 22 September 2026].

Milic, T., Stojanov, Z. & Vecstejn, I., 2025. APPLICATION OF THE STRIDE FRAMEWORK IN A MICROSERVICE. *JOURNAL OF ENGINEERING MANAGEMENT AND COMPETITIVENESS (JEMC)*, 15(2), p. 100.

Odera, D., Otieno, M. & Ounza, J. E., 2023. Security risks in the software development lifecycle: A review. *World Journal of Advanced Engineering Technology and Sciences*, 8(2), p. 1.

OWASP Foundation, n.d. *Secrets Management Cheat Sheet*. OWASP Cheat Sheet Series. Available at: [OWASP Secrets Management Cheat Sheet](#) [Accessed 22 September 2026].

Tchami, L., 2023. *Dependency Vulnerability Checking*. [Online]
Available at: <https://www.linkedin.com/pulse/dependency-vulnerability-checking-lionel-tchami-nfada-bsc-msc-/>
[Accessed 25 Sept 2026].

Verdugo, J., Oviedo, J., Rodríguez, M. & Piattini, M., 2025. Experiences with requirements in an accredited laboratory for software and data quality evaluation. In *2025 IEEE 33rd International Requirements Engineering Conference*, pp. 378-388.

Washington, G., 2025. *Secure Design vs Secure Coding*. [Online]
Available at: <https://www.linkedin.com/pulse/secure-design-vs-coding-washington-gomez-ordoqui-0jk4f/>
[Accessed 25 Sept 2026].

Wiersema, L., 2025. Incorporating Threat Modelling into Security by Design. *Utrecht University Repository*, p. 1.

AI RESEARCH AND VERIFICATION RECORD

Team Member	AI/Tool Use	Purpose	Output Used?	How Independently Verified?	What was changed/rejected/accepted ?
Milenke Field	ChatGPT	Used to get assignment layout and direct research for Q2	Partially yes	Compared with other academic/standards sources, recognised professional resources, and official technological Documentation to check for alignment	some content was altered and merged where facts aligned with academic/standards sources, recognised professional resources, and official Technological documentation
Chanice Grant	ChatGPT	Used to help structure Q1, clarify QA/QC/V&V, risk-based verification, verification evidence, automation limitations and the Risk-to-Verification Evidence Map	Partially	Sources were selected and accessed independently and incorporated as academic evidence rather than citing AI.	Unsupported or overly project-specific statements were removed or changed. Final CivicConnect M3 verification decisions were deliberately not made in Assignment 3.
Megan Wathen	ChatGPT	Used for grammar and spell checking	Yes	I reread everything and had an external party read my section.	Some of my sentences were not as formal as it should be.